

Практическая работа 5 по Информационным технологиям

Стек. Системные вызовы.

Для динамического хранения локальных переменных и адресов возврата нам нужен стек.

Абстракция «стек»

Хранилище «объектов»

- Основные операции — положить на стек, снять со стека
- Основной доступ — последовательный, к верхнему элементу стека
 - Иногда разрешается относительный доступ к N-му элементу стека от вершины
- ⇒ «Первым вошёл — последним вышел»
- Может расти бесконечно
- Поведение при исчерпании (снятии с пустого стека) не определено

Реализация стека в машинных кодах

- «Объект» — слово (word, то есть 4 байта или 32 бита);
- Хранилище — область памяти, ограниченная только с одной стороны («дно» стека), а с другой — «бесконечно» растущая (пока не достигнет занятой области памяти);
- Рост/снятие: специальная ячейка памяти, хранящая адрес вершины стека + знание адреса дна стека:
 - Используется косвенная адресация;
 - ⇒ Удобно (на некоторых архитектурах — единственно возможно) хранить в регистре;
 - Переполнение (попытку положить на стек больше значений, чем предусмотрено конвенцией) надо как-то проверять;
 - Исчерпание (попытку снять со стека больше значений, чем там есть в данный момент) тоже надо как-то проверять.

Возможная аппаратная поддержка стека

- Атомарные операции добавления/снятия (в обычном случае действия два: чтение/запись значения и изменение указателя)
 - например, автоувеличение/автоуменьшение указателя прямо в процессе взятия значения;
 - в случае программного стека, как в RISC-V, эта операция не такая уж частая. На стек кладётся обычно не одно значение, а несколько, так что эффективнее сначала изменить указатель сразу на требуемое под все значения смещение, а затем заполнить память значениями.
- Двойная косвенная адресация позволяет напрямую обращаться к данным, *адреса* которых лежат в стеке;
 - соответствует использованию указателей в Си, и сильно упрощает *программисту* реализацию любых структур с указателями (например, связанных списков);
 - в случае RISC-V — недопустимо долгое двойное обращение к памяти.

Реализация стека в Ripes

...регулируется соотв. конвенцией:

- Выделенный регистр **sp (x2)**, stack pointer, указатель стека, вершина стека.
- Дно стека (bottom) — `0x7fffffff` (непосредственно под областью ядра).
- Стек растёт *вниз* по одному слову (4 байта).
- Граница роста стека — область кучи (`0x10040000` или выше, если куча выросла), определяется вручную или ядром операционной системы.
- Операции добавления и снятия — *неатомарные*:
 - при добавлении сначала уменьшаем указатель, затем записываем значение;
 - при снятии сначала считываем значение, затем увеличиваем указатель.

Пример использования стека:

```
.text
main:
    li t1 123          # какое-то значение - в t1
    addi sp sp -4      # положить на стек - 1
    sw t1 0(sp)        # положить на стек - 2
    addi t2 t1 100     # какое-то ещё значение
    addi sp sp -4      # положить на стек - 1
    sw t2 0(sp)        # положить на стек - 2
    lw t3 0(sp)        # доступ к вершине стека
    lw t4 4(sp)        # доступ ко 2-му элементу стека
    lw t0 0(sp)        # снять со стека - 1
    addi sp sp 4        # снять со стека - 2
    addi sp sp -4      # положить на стек - 1
    sw zero 0(sp)      # положить на стек - 2
    addi a7, zero, 10   # halt
    ecall
```

В кодах и после дизассемблера программа выглядит без изменений, потому что здесь почти нет псевдоинструкций:

Address	Code	Basic	Line	Source
0x00400000	0x07b00313	addi x6,x0,0x0000007b	1	li t1 123 # какое-то значение
0x00400004	0xffc10113	addi x2,x2,0xffffffffc	2	addi sp sp -4 # положить на стек - 1
0x00400008	0x00612023	sw x6,0(x2)	3	sw t1 (sp) # положить на стек - 2
0x0040000c	0x06430393	addi x7,x6,0x00000064	4	addi t2 t1 100 # какое-то ещё значение
0x00400010	0xffc10113	addi x2,x2,0xffffffffc	5	addi sp sp -4 # положить на стек - 1
0x00400014	0x00712023	sw x7,0(x2)	6	sw t2 (sp) # положить на стек - 2
0x00400018	0x00012e03	lw x28,0(x2)	7	lw t3 (sp) # доступ к вершине стека
0x0040001c	0x00412e83	lw x29,4(x2)	8	lw t4 4(sp) # доступ ко 2-му элементу стека
0x00400020	0x00012283	lw x5,0(x2)	9	lw t0 (sp) # снять со стека - 1
0x00400024	0x00410113	addi x2,x2,4	10	addi sp sp 4 # снятие со стека - 2
0x00400028	0xffc10113	addi x2,x2,0xffffffffc	11	addi sp sp -4 # положить на стек - 1
0x0040002c	0x00012023	sw x0,0(x2)	12	sw zero (sp) # положить на стек - 2

Запустите этот пример в Ripes и пройдите его по шагам. Чтобы посмотреть ячейку памяти, соответствующую вершине стека, перейдите на страницу **Memory** и установите адрес 0x7ffffffc в окне Go to section.

Операция доступа к N-му элементу стека полностью аналогична доступу к вершине стека (просто смещение не нулевое, а $+N*4$). Наверное, ещё и поэтому стек растёт вниз — смещения проще читать.

Память, которую некоторое время занимал стек, а затем указатель ушёл выше, продолжает хранить значения, которые там лежали. Однако надеяться на то, что они там пребудут впредь, нельзя: память считается свободной и её, как минимум, меняют последующие добавления в стек.

Конвенция устроена так, что все критичные данные постоянно находятся внутри стека (между началом и вершиной). Если, допустим, при снятии сначала изменять указатель, а только потом считывать значения, возникнет опасная ситуация: в это время сохранённые значения оказываются в свободной памяти, и их может испортить кто угодно — например, обработчик прерывания.

Таким образом, хранение данных в стеке:

- эффективней, чем в произвольном месте памяти (lw/sw не превращаются в псевдоинструкции), впрочем, это верно не только для стека, но и при адресации относительно регистра с фиксированным значением (например, при доступе к глобальным переменным относительно gp);
- использует адресацию относительно постоянно меняющегося регистра **sp**; как следствие, это требует аккуратного подсчёта текущей глубины стека;
- не требует явного указания адреса и заведения метки в программе на языке ассемблера;
- может привести к сбоям в работе при переполнении/исчерпании/неаккуратном использовании стека.

Системные вызовы

Имеется 8 системных инструкций:

- шесть считывают и записывают данные в регистры управления и состояния (control and status registers, CSR) системы (не рассматриваются)
- ecall выполняет системный вызов. Регистры, используемые для передачи параметров в вызов и возврата из него, определяются с помощью интерфейса ABI.
- ebreak устанавливает контрольную точку для отладчика. Вставляется вручную при написании программ и редко используется.

ecall — это обращение к ядру ОС, т.е. программа запрашивает какие-то данные из «окружения», в котором она выполняется.

Системный вызов — это вызов подпрограммы, в котором вместо адреса используется заданный в API/ABI (Application Binary Interface) номер системного вызова.

Вызов ecall в RISC-V (привязка к Ripes):

- в регистр a7 помещается номер системной функции;
- если есть параметр системного вызова, то он помещается в регистр a0
- инструкция ecall передаёт управление ОС (обычно ядру);
- возврат из системного вызова — по аналогии с возвратом из подпрограммы, на следующую после ecall инструкцию;

– возвращаемое значение (если есть) помещается в a0 в соответствии с работой вызванной системной функции.

В таблице 1 приведены некоторые системные вызовы, поддерживаемые Ripes.

Таблица 1.

a7	a0	Name	Description
1	(integer to print)	PrintInt	Prints the value located in a0 as a signed integer
4	(pointer to string)	PrintString	Prints the null-terminated string located at address in a0
10	-	Exit	Halts the simulator
11	(char to print)	PrintChar	Prints the value located in a0 as an ASCII character
30		Time_msec	Get the current time since epoch (milliseconds since 01.01.1970)
31		Cycles	Get number of cycles elapsed since program start
57		Close	Close a file
63		Read	Read from a file descriptor into a buffer
64		Write	Write to a file descriptor from a buffer
93	(status code)	Exit2	Halts the simulator and exits with status code in a0
1024		Open	Open a file from a path

Следующий пример программы читает строку текста из памяти и выводит её на экран консоли:

```
.data
str1: .asciz "Palindrom"
.text
# Palindrom
la t1, str1          # читаем адрес начала строки
Lp: lb s3, 0(t1)      # читаем один символ строки
    beq s3, zero, Lquit # достигнут конец строки?
    add a0, s3, zero    # готовим символ для печати
    addi a7, zero, 11   # готовим сист. вызов печати символа
    ecall              # системный вызов
    addi t1, t1, 1      # следующий индекс
    jal Lp              # переход в начало цикла Lp
Lquit: addi a7, zero, 10 # готовим системный вызов
      ecall            # Halt - Останов
```

В теле цикла мы использовали системный вызов для вывода содержимого регистра a0 на консоль. В результате программа сначала последовательно напечатает строку символов, а затем выполнит так называемый Останов с кодом 0.

Задание

Напишите на ассемблере RISC-V программу для вывода палиндрома - строки символов сначала в прямом порядке, а затем – в обратном. Используйте стек. Не забывайте писать комментарии. Строка символов задается переменной str1 в сегменте .data памяти.

Вариант палиндрома для нечётных номеров: No lemon, no melon

Вариант палиндрома для чётных номеров: Never odd or even

В результате программа должна напечатать:

No lemon, no melon

nolem on ,nomel oN

или, другой вариант:

Never odd or even

neve ro ddo reveN

Контрольные вопросы:

1. С каким шагом надо изменять значение указателя стека?
2. Существует ли механизм автоматического ограничения значения указателя стека?
3. В какую сторону надо изменять значение указателя стека при сохранении данных?